

Claude Code流出から見た、賢いモデルより失敗と自律性を運用できる組織

Claude Code流出から見た、賢いモデルより失敗と自律性を運用できる組織

本記事は、筆者とAIの対話を通じて生成されました。記事中の考察・構成・文章は、筆者の問いと判断を起点とし、AIによって補完・生成されたものです。事実関係の正確性については筆者が確認していますが、解釈および見解はAIとの共同思考の産物であり、いずれか一方に帰属するものではありません。

導入：事件の概要と「なぜこれを読む必要があるか」

2026年3月31日、AnthropicのAIコーディングツール「Claude Code」の全ソースコード51万2千行が、npmパッケージの設定ミスで世界に流出しました。数時間でGitHubにミラーされ、数万人の開発者が解析しました。

この記事はその技術的な詳細を語るものではありません。流出したコードを読んで見えてきた、**AIプラットフォームが本当に何を競争優位としているか**——そしてそれが日本企業のAI戦略に何を問いかけるかを論じます。

第一章：設計図は流出した。しかし「なぜそうなったか」は流出していない

コードを読んだ開発者たちが最初に驚いたのは、その複雑さではなく、**設計の思想的統一性**でした。

51万行を貫く一つの哲学があります。

「AIは自分を信用するな。しかし止まるな。」

懐疑的メモリ設計、自己修復ループ、バックグラウンドでの記憶整理——これらは机上では生まれません。**LLMが失敗する場面を何度も経験した人間が、必然的に到達する設計**です。

そして流出しなかったものがあります。Capybara（Claude 4.6の内部コード名）v4からv8にかけて虚偽主張率が悪化した記録、140バージョン以上にわたるシステムプロンプトの変更履歴、GrowthBookに蓄積されたA/Bテストの結果——**「なぜこの設計に至ったか」という経路**は、どこにも出ていません。

コードは地図です。しかし差別化の本質は、地図の描き方を学ぶ過程にあります。

第二章：コードの中に見える「設計の痕跡」が語るもの

※ここがブログ全体の核心



図は6層構造になっています。Layer 0が開発プロセスそのものの自己言及性、Layer 1・2が設計思想の核心、Layer 3・4が自律性の実装、Layer 5・6が競合対策と実験文化です。以下の各痕跡はこの層に対応しています。

痕跡1：失敗から生まれた「懐疑的メモリ」――図のLayer 1

通常のRAG実装は「取得して信じる」です。Claude Codeは違います。

メモリの中核は `MEMORY.md` という軽量のポインタ索引です。1行あたり約150文字のポインタだけを常時コンテキストに保持し、実際のプロジェクト知識はトピックファイルに分散させています。そして過去のトランスクリプトは決してコンテキストに全文読み込まれません。必要な識別子を `grep` で検索するだけです。

なぜRAGを使わないのか。コードのコメントが答えています――「自分の記憶をヒントとして扱い、必ずコードベースで検証せよ」。ベクトル検索で「それっぽいもの」を引いてくるのではなく、実際のファイルを確認するまで信じない。

さらに `autoDream` サービスがバックグラウンドで夜間に動き、その日の記憶を整理します。矛盾を除去し、曖昧な情報を確定情報に変換し、不要なものを刈り込む。LLMが自分の記憶を過信して誤った判断をし続けた失敗の積み重ねからしか生まれない設計です。

また、書き込みには「Strict Write Discipline」と呼ばれる規律があります。ファイルへの書き込みが成功した後にのみインデックスを更新できる。失敗した試みでコンテキストを汚染しない――この一点に、何度も痛い目を見た組織の経験が凝縮されています。

痕跡2：閾値「15秒」という経験値――図のLayer 3

`KAIROS` という自律エージェントモードがあります。ユーザーの入力を待たず、15秒ごとに `<tick>` プロンプトを受け取り「今何かすべきか？」を自律判断します。エラーを修正し、ファイルを更新し、タスクを実行する――すべてユーザーが何も言わなくても。

ここに一つの制約があります。

「ユーザーのワークフローを15秒以上ブロックする先行アクションは延期する」

この数字に理論的根拠はありません。実際に使って「これくらいだと邪魔に感じる」という体験データから来ています。さらに、KAIROSが起動しているときはレスポンスが `Brief`（極めて簡潔）モードに切り替わります。常駐アシスタントがターミナルを文字で埋め尽くさないための配慮です。

数字と細部が経験を証明しています。こういう閾値は、実際に使って迷惑をかけた後でしか決まりません。

痕跡3：バージョン管理されたプロンプト――図のLayer 2・6

多くの開発者が「Claude Codeのシステムプロンプト」と聞いて想像するのは、一本の長い文字列でしょう。実態は違います。

システムプロンプトは110以上の断片が動的に合成されています。ツールごと、サブエージェントごと、環境ごとに専用のプロンプトが存在し、状況に応じて組み合わせられます。さらに `SYSTEM_PROMPT_DYNAMIC_BOUNDARY` という境界を設け、静的な部分はキャッシュし、動的な部分だけを毎回生成することでAPIコス

トを制御しています。

そしてPiebald-AIが記録した140バージョン以上の変更履歴。これは**「プロンプトをコードと同様に実験・管理する」という組織文化**の証拠です。「なんとなく直した」ではなく、何を変えたら何が改善・悪化したかをバージョンで追跡できる状態で運用しています。

多くの企業がプロンプトを「ドキュメントフォルダのテキストファイル」として管理しているのとは、根本的に発想が異なります。

痕跡4：自己言及的な開発ループ――図のLayer 0

`undercover.ts`（約90行）というファイルがあります。Anthropic社員がClaude Codeを使って外部のオープンソースリポジトリにコミットする際に起動し、コミットメッセージやPRに以下を含めることを禁止します――内部コードネーム（Capybara、Tengu等）、未リリースのモデルバージョン番号、「Claude Codeを使っている」「自分がAIである」という事実、Co-Authored-Byの帰属表記。

このモードはオフにできません。`CLAUDE_CODE_UNDERCOVER=1` でオンを強制できますが、オフを強制する環境変数は存在しない一方通行の設計です。

つまり、Anthropic自身がClaude Codeを使ってClaude Codeを開発しています。これはオプションではなく、強制インフラとして組み込まれています。ツールを作るプロセス自体が、すでにそのツールに依存している――自己言及的なループが製品の中に構造として埋め込まれています。

第三章：これは「誰が設計したのか」という問いを無効化する

委員会設計ではありません。思想的―貫性がそれを否定しています。

では誰が設計したのか。コードの中に答えがあります。

`coordinatorMode.ts` というファイルには、こんな運用原則が埋め込まれています。

「Parallelism is your superpower。独立したタスクは並列に投げろ。調査は多角的にfan-outしろ。実装と検証は分離しろ。」

さらに重要なのは次の制約です。coordinatorは、workerの結果をそのまま上位に再委譲してはならない。必ず自分でfindingsを理解し、synthesized specに変換してから渡せ、と明記されています。

これは設計の美学ではありません。AI同士の分業で品質が崩壊するポイントを、実際に何度も踏んだ後でしか書けないルールです。「誰かが頭の中で考えた」のではなく、壊れてから書き足された一文です。

こういう複雑系が成立する組織は、おそらく3層に分かれています。

- 第一層：核心の発想を作る少数の設計者――「AIは自分を信用するな」という原則を最初に置いた人間
- 第二層：失敗を測る評価基盤チーム――Capybara v4→v8の虚偽主張率の変化を記録し続けた人間
- 第三層：実運用で壊れ方を潰す実装チーム――coordinatorのfan-out、permissionのclassifier、15秒の閾値を決めた人間

この3層は会議で整合したのではありません。小さな原型を運用にさらし、壊れた箇所を測り、増築し、規律化した結果として成立しています。設計というより、進化に近いです。

ではこの複雑系を、なぜ破綻なく動かせるのか――評価は「誰が」やっているのか

ここで実務家が当然抱く疑問があります。チーム内の設計と実装の境界が溶け、AIがAIを管理し、数百万ユーザーが使うシステムとして、なぜこれが破綻しないのか。そして評価は誰がやっているのか。

コードを読んだ答えは、ある意味で不安を深めるものです。評価もまた、AIがやっています。ただし、その構造には注目すべき設計思想があります。

流出コードの中に `yoloClassifier.ts` という1,495行のファイルがあります。これが「autoモード」の権限判定エンジンです。名前の「yolo」は内部の冗談でしょうが、その中身は真剣です。

このclassifierがやっていることは、単純な許可リストの照合ではありません。ツール呼び出しのたびに、別のSonnet 4.6モデルを呼び出して、その操作がユーザーの意図と一致しているかどうかを独立に判定します。つまり、Claudeの判断をClaude（別インスタンス）が評価するという構造です。

さらに重要な設計上の工夫があります。このclassifierには、Claudeが読んだファイルやWebページの内容が一切渡されません。ユーザーのメッセージとツール呼び出しだけが入力です。これにより、悪意あるコンテンツがファイルに埋め込まれていても、classifierがそれに操作されることを防いでいます。評価者を意図的に「情報から切り離す」ことで、独立性を担保しています。

サブエージェントに対してはさらに厳しい評価が走ります。サブエージェントが起動するとき、classifierは委譲タスクを事前評価します。そしてサブエージェントが終了したとき、classifierはそのエージェントの全行動履歴を事後レビューします。起動時には安全だったエージェントが、途中で読んだコンテンツによって動作を変えた可能性があるからです。

`bashSecurity.ts` は2,592行、23の番号付きセキュリティチェックを持ちます。Zshのイコール展開によるcurl迂回、ゼロ幅スペースのインジェクション、ANSI-Cクォーティングによる任意文字エンコード――これらのチェックは理論から来ていません。23の番号付きチェックそれぞれが、実際のインシデントの存在を示唆しています。誰かが実際にやられた後に書き足された一行ずつです。

そしてコードの内部コメントにこういう記述があります。

「BQ 2026-03-10: 1,279セッションがセッション内で50回以上連続失敗（最大3,272回）し、世界全体で1日約25万回のAPIコールを無駄にしている」

この観測に対する修正コードは3行です。`MAX_CONSECUTIVE_AUTOCOMPACT_FAILURES = 3`――3回連続で失敗したら、そのセッションでの自動コンパクションを無効化する。25万回/日の無駄なAPI呼び出しを、3行のコードで止めた。この種の修正は、実際のトラフィックデータを継続的に監視していなければ気づけません。

つまり評価の構造はこうなっています。Claudeがアクションを起こす→別のClaudeが事前・事後に評価する→実際のセッションデータから異常を検知する→コードで修正する――この4段階のループが常時回っています。

評価は人間がやっているのではありません。しかし、どの数字を評価指標にするかは人間が決めています。そしてその「どの数字を見るか」を間違えると、25万回/日のコストを垂れ流し続けることになる。ここが本当に難しい部分です。

第四章：日本企業で再現可能な部分と、そうでない部分

領域	再現可能性	ボトルネック
懐疑的AI運用の原則	今日から	認識の問題
プロンプトの分割・バージョン管理	今日から	習慣の問題
失敗ログの体系的な蓄積	今日から	文化の問題
AIによる内製開発ループ	数ヶ月～1年	意思決定と人材
自前の評価インフラ構築	1～3年	技術と時間
ユーザー規模でのA/B実験	市場獲得が先	市場の問題
アーキテクト人材の密度	5～10年単位	人材市場の構造

最も重要な示唆は、**上位3つが「お金でも技術でもなく、認識と習慣の問題」**だということです。

多くの日本企業がAIを導入しながら、AIの出力を疑わず、プロンプトをバージョン管理せず、失敗を記録していません。ここから始めることが、frontier labとの差を縮める唯一の現実的な経路です。

結論：問われているのは、組織の能力である

今回の流出を読んで、率直に言えばこう感じます。Anthropicというfrontier labは、組織としてとんでもないところに行っています。

機械学習の素養がある開発者ほど、この感覚は強いはずです。なぜなら、コードを読めば読むほど明らかになるからです――これはモデルが優秀なだけでは、できない。

これからのエージェント開発で問われるのは、どれだけ賢いモデルを持っているかではありません。

どれだけ複雑な自律性を、壊れ方まで含めて評価し、運用し、改善できる組織かです。

Claude Codeの流出が教えてくれたのは、競合の技術ではありません。**自分たちの組織に、その能力があるかどうかを問う視点**でした。

これからのAIEージェント開発にも有用な視点を提供してくれた事件だったように思います。